RAG

Retrieval Augmented Generation

Objectifs du cours

À la fin de cette séance, vous devriez être capables de :

- expliquer l'architecture d'un système de génération augmentée par récupération (RAG) ;
- transformer un corpus documentaire en index exploitable ;
- comprendre les choix de découpage, d'embeddings et de recherche ;
- améliorer la pertinence des passages récupérés ;
- évaluer un RAG indépendamment du modèle génératif ;
- diagnostiquer les erreurs et proposer des améliorations.

Objectif

L'objectif n'est pas seulement de faire fonctionner un RAG, mais de comprendre pourquoi il fonctionne – ou échoue.

Déroulé du CM

1. Architecture générale d'un RAG
2. Préparation documentaire et chunking
3. Embeddings et bases vectorielles
4. Retrieval, recherche hybride et reranking
5. Construction du contexte et génération
6. Évaluation et diagnostic
7. Démonstration et architectures avancées

Fil rouge

Construire un assistant capable de répondre à des questions à partir d'un corpus documentaire, en retrouvant les passages pertinents et en justifiant ses réponses.

Pourquoi un RAG ?

Un LLM seul présente plusieurs limites :

- ses connaissances peuvent être incomplètes ou obsolètes ;
- il ne connaît pas nécessairement les documents privés de l'organisation ;
- il peut produire une réponse plausible mais non étayée ;
- réentraîner ou fine-tuner le modèle n'est pas toujours adapté à une mise à jour documentaire.

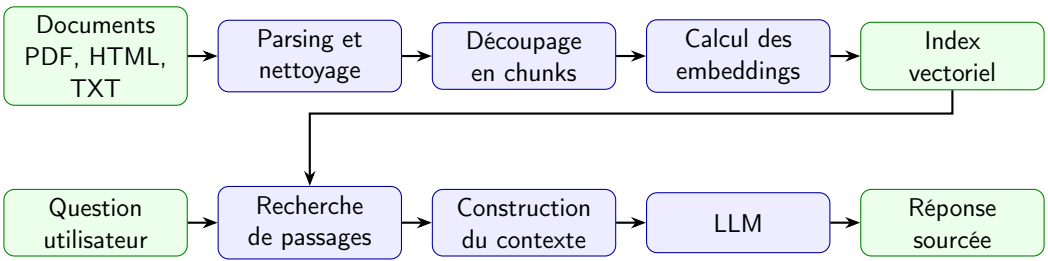
Question centrale

Comment donner au modèle accès à des connaissances externes, sans réentraîner le modèle à chaque modification du corpus ?

Idée clé

Le RAG combine une recherche documentaire et un modèle génératif : le modèle répond à partir de passages récupérés au moment de la question.

Architecture générale d'un RAG



Deux pipelines complémentaires

Indexation : préparer et indexer les documents.
Inférence : retrouver les informations utiles, puis générer la réponse.

Indexation et inférence : deux problèmes différents

Pipeline d'indexation

1. Collecter les documents
2. Extraire le texte
3. Nettoyer et découper
4. Calculer les embeddings
5. Stocker les vecteurs et métadonnées

Pipeline de requête

1. Recevoir la question
2. Transformer la requête si nécessaire
3. Récupérer les passages
4. Filtrer et réordonner
5. Construire le contexte
6. Générer la réponse

Idée clé

Une mauvaise indexation pénalise toutes les questions. Une mauvaise stratégie de recherche pénalise les questions concernées, même avec un bon corpus.

Étape 1 – Préparer les documents

Les documents bruts ne sont pas toujours directement exploitables.

- **Parsing :** extraire le texte depuis PDF, HTML, DOCX ou pages web.
- **Nettoyage :** supprimer les éléments parasites, en-têtes répétés et contenus dupliqués.
- **Structure :** préserver titres, sections, tableaux et listes lorsque c'est utile.
- **Métadonnées :** conserver la source, la date, la section et les droits d'accès.

Exemple de métadonnées

```
{source: "guide.pdf", section: "Installation", page: 12}
```

Question centrale

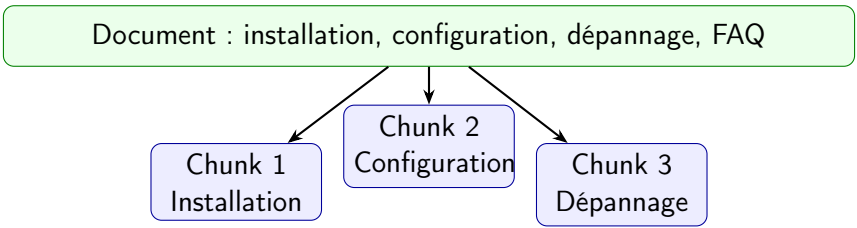
Si la structure ou la provenance disparaît pendant l'extraction, comment retrouver ensuite le contexte d'un passage ?

Étape 2 – Pourquoi découper en chunks ?

Un document entier peut être trop long ou trop peu précis pour la recherche.

- Le modèle d'embeddings possède une limite de taille d'entrée.
- Un document long mélange souvent plusieurs sujets.
- Le retrieval doit pouvoir sélectionner une portion pertinente.
- Le contexte fourni au LLM doit rester suffisamment compact.

Étape 2 – Pourquoi découper en chunks ?



Idée clé

Le chunking n'est pas un simple prétraitement : il détermine quelles unités de connaissance pourront être retrouvées.

Stratégies de chunking

Stratégie	Principe	Limite principale
Taille fixe	Découper tous les N caractères ou tokens	Peut couper une idée en deux
Structurel	Respecter titres, paragraphes, sections	Sections de tailles très variables
Sémantique	Détecter les changements de sujet	Plus complexe, dépend de la méthode
Hiérarchique	Retrouver un petit passage et récupérer son contexte parent	Gestion plus complexe de l'index

Paramètres à tester

Taille des chunks, chevauchement (*overlap*), respect de la structure et conservation des métadonnées.

Question centrale

Un chunk plus petit améliore-t-il toujours la qualité du RAG ?

Choisir la taille des chunks

Chunks trop petits

- Contexte incomplet
- Définitions séparées de leurs conditions
- Davantage de passages à agréger

Chunks trop grands

- Plusieurs sujets dans un même vecteur
- Passages moins ciblés
- Budget de contexte consommé rapidement

Idée clé

Il n'existe pas de taille optimale universelle. La bonne configuration dépend du corpus, des questions et du modèle utilisé.

En pratique : commencer par un découpage structurel, mesurer les résultats, puis comparer plusieurs tailles et chevauchements.

Étape 3 – Embeddings et similarité

Un modèle d'embeddings transforme un texte en vecteur numérique :

$$f(\text{texte}) = \mathrm{v} \in \mathbb{R}^d$$

Les textes sémantiquement proches devraient avoir des représentations proches dans l'espace vectoriel.

Similarité cosinus

$$\mathrm{sim}(\mathrm{u}, \mathrm{v}) = \frac{\mathrm{u} \cdot \mathrm{v}}{\|\mathrm{u}\| \, \|\mathrm{v}\|}$$

- On calcule les vecteurs des chunks à l'indexation.
- On calcule le vecteur de la question à la requête.
- On recherche les vecteurs les plus proches.

Idée clé

La proximité vectorielle est un signal de pertinence, pas une preuve que le passage répond correctement à la question.

Modèles d'embeddings : que faut-il regarder ?

Le choix du modèle influence directement la qualité du retrieval.

- Langues prises en charge, notamment le français.
- Domaine du corpus : généraliste, technique, scientifique, etc.
- Taille des vecteurs et coût mémoire.
- Latence d'encodage et de recherche.
- Compatibilité entre embeddings de requêtes et de documents.

Outil de référence pour le TD

`sentence-transformers` permet d'encoder des textes avec des modèles préentraînés.

```
model = SentenceTransformer(
"sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2" )
texts = [ "Installer le client Python", "Configurer la connexion au serveur" ]
vectors = model.encode(texts, normalize_embeddings = True)
```

Pourquoi utiliser une base vectorielle ?

Une base vectorielle permet de stocker et rechercher des vecteurs associés à des informations documentaires.

- Identifiant du chunk
- Vecteur d'embedding
- Texte du chunk
- Métadonnées : source, section, date, catégorie, droits d'accès

Technologies

- **Qdrant :** base vectorielle avec filtres, métadonnées et service de recherche.
- **FAISS :** bibliothèque de recherche de similarité vectorielle, utile pour des expérimentations locales.

Idée clé

Pour le TD et le TP, nous utiliserons Qdrant comme référence. FAISS peut servir de comparaison, sans imposer deux implémentations complètes.

Indexation et recherche : exemple avec Qdrant

Le principe consiste à associer chaque vecteur à un identifiant et à une charge utile (*payload*).

```
client = QdrantClient("memory")
client.create_collection(collection_name = "documents", vectors_config =
models.VectorParams(size = 384, distance = models.Distance.COSINE))
Chaque point contient un vecteur et ses metadonnees. client.upsert(
collection_name = "documents", points = [models.PointStruct(id = 1, vector =
vectors[0].tolist(), payload = "text" texts[0], "source"" guide.pdf")])
::
```

La dimension du vecteur doit correspondre à celle produite par le modèle choisi. Le stockage du texte et des métadonnées facilite l'inspection des résultats.

Étape 4 – Retrouver les bons passages

Le *retrieval* sélectionne les passages susceptibles de répondre à la question.

1. Encoder la question.
2. Comparer son vecteur à ceux de l'index.
3. Sélectionner les k passages les plus proches.
4. Récupérer le texte et les métadonnées associés.

Le paramètre top-k

`top_k = 5` demande par exemple les cinq premiers résultats.

- k trop petit : risque de manquer une information utile.
- k trop grand : davantage de bruit et de contexte à traiter.

Question centrale

Les passages retrouvés répondent-ils vraiment à la question, ou parlent-ils simplement d'un sujet similaire ?

Recherche vectorielle : limites

La recherche vectorielle est utile pour la similarité sémantique, mais elle n'est pas parfaite.

- Les identifiants, codes d'erreur et références exactes peuvent être mal retrouvés.
- Une formulation proche peut masquer une différence importante.
- Le meilleur résultat peut ne contenir qu'une partie de la réponse.
- Un passage très similaire peut être obsolète ou hors sujet.

Exemple

Question : « Comment résoudre l'erreur E1042 ? »
Un passage qui décrit un problème voisin peut être sémantiquement proche sans mentionner le code E1042.

Idée clé

La qualité d'un RAG dépend autant de la stratégie de recherche que de la qualité des embeddings.

Recherche lexicale, vectorielle et hybride

Approche	Atouts	Limites
Lexicale (BM25)	Mots exacts, codes, noms propres	Synonymes et reformulations
Vectorielle	Similarité sémantique, reformulations	Confusions entre concepts proches
Hybride	Combine signaux lexicaux et sémantiques	Fusion et réglage supplémentaires

Fusion des résultats

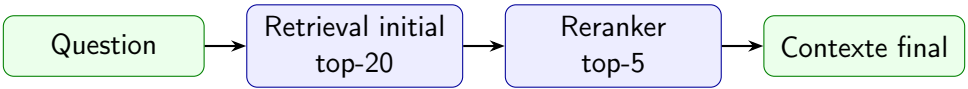
Une méthode comme Reciprocal Rank Fusion (RRF) combine les classements produits par plusieurs systèmes.

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{c + \mathrm{rang}_r(d)}$$

L'approche hybride est particulièrement intéressante lorsque le corpus contient à la fois du texte courant, des noms propres et des identifiants techniques.

Reranking : réordonner les candidats

Le retrieval initial privilégie souvent la rapidité. Un reranker peut ensuite réévaluer les candidats les plus prometteurs.



- Un modèle de type *cross-encoder* évalue directement la paire question–passage.
- Il peut mieux distinguer les passages réellement pertinents.
- Il est généralement plus coûteux qu'une simple recherche vectorielle.

Idée clé

Stratégie fréquente : récupérer largement, puis reranker pour réduire le bruit.

Transformer la question pour mieux chercher

La formulation initiale de l'utilisateur n'est pas toujours adaptée à la recherche documentaire.

- **Réécriture :** reformuler une question ambiguë.
- **Multi-query :** générer plusieurs formulations complémentaires.
- **Décomposition :** séparer une question complexe en sous-questions.
- **Filtres :** restreindre la recherche par date, source ou catégorie.

Exemple

Question : « Est-ce que je peux l'utiliser avec notre version actuelle ? »
La recherche peut nécessiter de résoudre le référent « l' », d'identifier la version du logiciel et de retrouver la documentation de compatibilité.

Attention : ces transformations ajoutent de la complexité et peuvent introduire de nouvelles erreurs. Elles doivent être évaluées sur des questions représentatives.

Du retrieval au contexte du LLM

Les passages récupérés ne doivent pas être simplement concaténés sans contrôle.

- Supprimer les doublons et les passages redondants.
- Limiter le nombre de passages et le budget de tokens.
- Conserver la provenance et les références des sources.
- Préserver les relations entre passages si elles sont nécessaires.
- Distinguer les instructions du système du contenu documentaire.

Exemple de consigne

Réponds à partir des extraits fournis. Si les informations sont insuffisantes, indique-le. Associe les affirmations importantes à leurs sources.

Idée clé

Le RAG ne garantit pas automatiquement une réponse exacte : le modèle peut ignorer, mal interpréter ou extrapoler à partir du contexte.

Réponse sourcée : que faut-il vérifier ?

Une bonne réponse

- Répond à la question posée
- Est soutenue par les passages
- Cite les sources utiles
- Signale les informations manquantes

Signaux d'alerte

- Citation inexistante
- Conclusion absente des sources
- Confusion entre versions
- Réponse affirmative malgré un contexte insuffisant

Question centrale

Comment distinguer un échec de recherche d'un échec de génération ?

Si le bon passage n'est pas retrouvé, il faut d'abord améliorer le retrieval. Si le passage est présent mais mal utilisé, il faut examiner le contexte et la génération.

Évaluer un RAG : ne pas se limiter à quelques exemples

L'évaluation doit séparer plusieurs dimensions.

1. **Retrieval** : les passages pertinents sont-ils retrouvés ?
2. **Génération** : la réponse est-elle correcte et utile ?
3. **Fidélité** : la réponse est-elle soutenue par le contexte ?
4. **Provenance** : les citations pointent-elles vers les bonnes sources ?
5. **Coût** : latence, consommation de tokens et coût des modèles.

Jeu de test

Préparer des questions représentatives, avec pour chacune une réponse attendue et, si possible, les passages documentaires pertinents.

Idée clé

Sans jeu de test, une modification du chunking ou du retrieval peut donner une impression d'amélioration sans preuve mesurable.

Quelques métriques de retrieval

Recall@k

Quelle proportion des passages pertinents est retrouvée parmi les k premiers résultats ?

$$\text{Recall@k} = \frac{\text{passages pertinents retrouvés dans le top-k}}{\text{nombre total de passages pertinents}}$$

Precision@k

Quelle proportion des k premiers résultats est pertinente ?

$$\text{Precision@k} = \frac{\text{passages pertinents dans le top-k}}{k}$$

MRR

À quel rang apparaît le premier résultat pertinent, en moyenne ?

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{\text{rang}_i}$$

Diagnostiquer les erreurs

Symptôme	Cause possible	Piste d'amélioration
Bon passage absent	Chunking ou retrieval inadéquat	Examiner BM25, embeddings, filtres et top-k
Passage pertinent mais incomplet	Découpage trop fin	Chunk plus large ou récupération du parent
Trop de bruit	Top-k élevé ou recherche imprécise	Filtrage, hybride, reranking
Réponse non étayée	Contexte mal exploité	Consigne, citations et règle d'abstention
Mauvaise version documentaire	Métadonnées ou corpus obsolètes	Filtrage temporel et gouvernance

Idée clé

Toujours inspecter les passages retrouvés avant de modifier le prompt ou le modèle génératif.

Démonstration fil rouge : construire un RAG

1. Charger un petit corpus documentaire.
2. Extraire le texte et conserver les sources.
3. Tester une stratégie de chunking.
4. Calculer les embeddings avec Sentence Transformers.
5. Indexer les vecteurs dans Qdrant.
6. Poser une question et inspecter les passages retrouvés.
7. Générer une réponse avec les sources.
8. Modifier un paramètre et comparer les résultats.

Ce qu'il faut observer pendant la démo

Les passages retrouvés, leur rang, leur provenance et la différence entre la réponse obtenue avec un retrieval naïf et une stratégie améliorée.

Expérience comparative : quel changement améliore le RAG ?

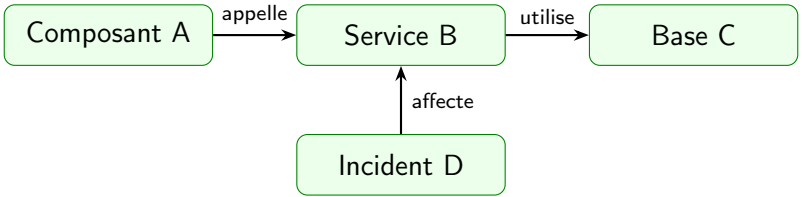
On compare les systèmes sur le même corpus et le même jeu de questions.

Configuration	**Changement**	**Question d'évaluation**
A. Baseline	Chunks fixes + recherche vectorielle	Les passages pertinents sont-ils retrouvés ?
B. Chunking amélioré	Découpage structurel	Le contexte est-il mieux préservé ?
C. Recherche hybride	BM25 + vecteurs	Les codes et termes exacts sont-ils mieux retrouvés ?
D. Reranking	Réordonnancement des candidats	Les premiers résultats sont-ils plus pertinents ?

Règle expérimentale : ne changer qu'un élément à la fois pour comprendre son effet.

Au-delà du RAG vectoriel : GraphRAG

Le RAG vectoriel classique récupère surtout des passages proches de la question.
GraphRAG explore une autre représentation : un graphe de connaissances composé d'entités et de relations.



- Utile pour des questions qui nécessitent de relier plusieurs entités.
- Peut exploiter des relations explicites et des informations globales.
- Nécessite une construction et une maintenance du graphe.

Idée clé

GraphRAG n'est pas automatiquement supérieur au RAG vectoriel : son intérêt dépend de la structure des connaissances et des questions posées.

Autre ouverture : vers des connaissances évolutives

Certaines approches explorent des bases documentaires plus dynamiques, des synthèses persistantes ou des structures de connaissances continuellement enrichies.

- Mise à jour des connaissances au fil des nouveaux documents.
- Organisation de synthèses et de liens entre informations.
- Conservation de la provenance et des versions.
- Recherche combinant passages, synthèses et relations.

Point de vigilance

Une synthèse persistante peut faciliter la recherche, mais elle peut aussi propager des erreurs ou devenir obsolète. La provenance, la mise à jour et la validation restent essentielles.

Ces architectures, dont les approches de type LLMWiki, constituent une ouverture : elles ne sont pas nécessaires pour réussir le premier système RAG.

Sécurité et gouvernance des connaissances

Un système RAG expose également de nouveaux risques.

- **Prompt injection documentaire :** un document peut contenir des instructions malveillantes destinées au modèle.
- **Droits d'accès :** le retrieval ne doit pas exposer des documents interdits à l'utilisateur.
- **Obsolescence :** des informations anciennes peuvent rester indexées.
- **Traçabilité :** il faut pouvoir identifier la source utilisée.
- **Confidentialité :** les documents et requêtes peuvent contenir des informations sensibles.

Idée clé

Les documents récupérés sont des données à interpréter, pas des instructions qui doivent remplacer les règles du système.

À retenir

1. La qualité du RAG commence par la qualité du corpus et de son indexation.
2. Le chunking définit les unités de connaissance récupérables.
3. Les embeddings et la base vectorielle ne suffisent pas à garantir la pertinence.
4. BM25, recherche hybride et reranking peuvent améliorer le retrieval.
5. Le contexte doit être assemblé avec soin et les réponses doivent être sourcées.
6. Un jeu de test permet de comparer les stratégies et de diagnostiquer les erreurs.

Question centrale

Comment vérifier que mon RAG améliore réellement mon assistant, plutôt que de simplement lui fournir davantage de texte ?

Transition vers le TD et le TP

TD – Retrieval engineering

- Chunking et métadonnées
- Embeddings
- Indexation Qdrant
- Recherche vectorielle
- Comparaison des résultats
- Introduction au reranking

TP – Assistant augmenté

- Choisir ou recevoir un corpus
- Brancher le retrieval à l'assistant existant
- Afficher les sources
- Tester plusieurs questions
- Diagnostiquer et améliorer

Stack de référence : Sentence Transformers + Qdrant + LLM